

CO4_AT1_INDUSTRY PROBLEM SOLVING TASK

NAME: HASMATH NISHA M

REGISTER NO:192524513

CSA1220-COMPUTER ARCHITECTURE

Question 1

1. Role of each memory level

- **Cache memory (CPU L1/L2/L3, and application-level caches such as Redis/Memcached):** Holds the hottest, most frequently requested data — best-selling product entries, session tokens, and shopping-cart state. Because access latency is measured in nanoseconds, caching this data removes repetitive round-trips to the database and absorbs the bulk of read traffic during flash-sale traffic spikes.
- **DRAM (main memory):** Acts as the working set for the application and database servers. It holds active database buffer pools, indexes, and in-flight transaction data. DRAM's microsecond-level latency and byte-addressability make it ideal for the query engine, join operations, and connection-level state that changes constantly but must remain instantly accessible.
- **Secondary storage (SSD/HDD arrays, distributed storage):** Provides the durable, high-capacity system of record — full product catalogs, customer records, and committed transaction logs. It is orders of magnitude slower than DRAM but is non-volatile and cost-effective per gigabyte, so it stores everything that must persist beyond a server restart.

2. Why the platform slows down at peak load

At peak load, thousands of concurrent read/write requests overwhelm the capacity of cache and DRAM tiers, forcing more requests to fall through to secondary storage. Disk-level I/O contention, lock contention on hot rows (e.g., inventory counters), and cold-cache misses after scaling events all compound to increase tail latency.

3. Proposed memory hierarchy

Tier	Technology	Data placed here	Purpose
L1 – Edge/CDN cache	In-memory edge cache	Static assets, catalog pages, images	Cuts latency for read-heavy browsing traffic before it reaches origin servers
L2 – Distributed cache	Redis / Memcached cluster (DRAM-based)	Hot product data, session state, cart contents, price lookups	Sub-millisecond reads; absorbs most repeated queries; reduces DB load
L3 – Database buffer pool	Server DRAM	Active indexes, recently accessed rows, query result sets	Keeps working set of the database in memory to avoid disk reads
L4 – Primary storage	NVMe SSD (transactional DB)	Full transaction records, order history, customer records	Durable, fast persistence for ACID-compliant writes

Tier	Technology	Data placed here	Purpose
L5 – Archival storage	HDD / object storage (e.g., S3-class)	Historical orders, logs, backups, analytics data	Low-cost, high-capacity long-term retention

4. How this design achieves the goals

- **Reduced access latency:** Multi-level caching (edge → distributed cache → DB buffer pool) means the majority of read requests are served from DRAM-speed tiers, not disk.
- **Improved data locality:** Frequently accessed catalog and session data is kept physically close to the compute layer (cache/DRAM), following the principle of temporal and spatial locality; only cold or rarely accessed data touches slow storage.
- **High concurrent transaction throughput:** Write-heavy operations (checkout, inventory decrement) use fast SSD-backed storage with write-behind caching and asynchronous replication, while read-heavy operations are offloaded almost entirely to cache tiers, allowing the system to scale reads and writes independently.
- **Additional recommendations:** Use cache invalidation/TTL policies to keep cache data consistent, horizontal sharding of the database to spread transaction load, and read replicas to further isolate read traffic from write traffic during peak periods.

Question 2

1. Suitability evaluation

Memory type	Speed	Volatility	Best suited for	Limitation
SRAM	Fastest (ns-level), highly predictable	Volatile	Real-time control loops, register/actuator state, interrupt handlers, PID control values	Low density, expensive, small capacity
DRAM	Fast (tens of ns) but needs refresh cycles	Volatile	Buffering camera image frames, sensor data streams, intermediate computation	Refresh cycles and bus contention can introduce timing jitter, hurting determinism
Non-volatile memory (Flash/NVRAM/FRAM)	Slower writes, fast reads	Non-volatile	Program/firmware code, calibration data, configuration parameters that must survive power loss	Higher write latency and wear (for Flash) makes it unsuitable for high-frequency updates

2. Real-time constraints to consider

- Predictability matters more than raw speed: memory access times must be deterministic, since a fast-but-variable-latency system can miss hard real-time deadlines.
- SRAM's lack of refresh cycles makes it far more deterministic than DRAM, which is critical for the innermost control loop.
- Non-volatile memory is essential so control programs and calibration data survive power interruptions without needing to reload from an external source, which would violate startup deadlines.

3. Proposed memory hierarchy

- **Tier 1 — On-chip SRAM (tightly-coupled memory / cache):** Stores the real-time control loop variables, actuator commands, and interrupt-service-routine data. Being on-chip and directly addressed (not just cached), it guarantees single-cycle, predictable access — essential for closed-loop motor and safety control running at kHz rates.
- **Tier 2 — DRAM (main working memory):** Buffers incoming camera frames and sensor telemetry streams, and hosts the non-time-critical parts of the control application (path planning, vision processing). A dedicated DMA channel moves data between DRAM and SRAM so the CPU isn't blocked waiting on transfers.
- **Tier 3 — Non-volatile memory (Flash/FRAM/NVRAM):** Stores the firmware/program code, machine calibration tables, and configuration that must persist across power cycles. Code can be shadow-copied into SRAM/DRAM at boot for fast execution, while NVM remains the authoritative, persistent copy.

4. Design principles for determinism

- Partition memory so time-critical tasks execute entirely from SRAM, avoiding cache-miss variability.

- Use DMA and double-buffering for sensor/image data so CPU processing overlaps with data transfer without blocking.
- Reserve NVM writes for infrequent events (configuration changes, logging) to avoid write-latency spikes during operation.
- Apply memory protection/partitioning so a runaway non-critical task cannot starve the real-time control loop of memory bandwidth.

Question 3

1. Impact of each factor on video editing performance

- **Cache size (CPU/GPU L1–L3):** A larger cache holds more of the active frame data, filter coefficients, and codec lookup tables close to the processor, reducing stalls during frame-by-frame effects rendering. However, 4K/8K frames (tens of MB per uncompressed frame) exceed cache capacity, so cache mainly accelerates the computation on the current frame/tile rather than holding whole video streams.
- **DRAM capacity:** Multiple 4K/8K streams, each requiring large uncompressed frame buffers for real-time preview, quickly consume system memory. Insufficient DRAM forces the editing software to page frames out to disk, causing stutter ("dropped frames") during scrubbing or playback of multi-track timelines.
- **Memory bandwidth:** 8K video at high frame rates demands very high sustained data-transfer rates between storage, DRAM, and the GPU. Bandwidth — not just capacity — becomes the bottleneck when compositing multiple streams, applying real-time color grading, or exporting.
- **Storage performance:** Video files are far larger than DRAM capacity, so continuous streaming from storage is required. Slow storage (HDD) cannot sustain the read throughput needed for uncompressed or lightly compressed 4K/8K, causing buffering delays; fast NVMe SSDs (with RAID striping for extreme cases) are needed to keep up.

2. Recommended hierarchical memory organization

Tier	Recommendation	Role
CPU/GPU cache	Large L2/L3 CPU cache; large GPU on-die cache	Accelerates per-frame computation: color transforms, effects, encode/decode
System DRAM	64–128 GB+, high-frequency dual/quad-channel	Holds decoded frame buffers for multiple timeline tracks and preview caches
GPU VRAM	High-bandwidth GDDR6/HBM, 16–24 GB+	Real-time rendering, GPU-accelerated effects and playback compositing
Fast local storage	NVMe SSD (PCIe 4.0/5.0) or RAID-0 NVMe array	Streams raw/compressed footage at sustained multi-GB/s rates
Bulk/archive storage	High-capacity HDD or network storage (NAS/SAN)	Stores raw footage libraries and completed project archives

3. How this reduces delays and improves throughput

- Wide, high-frequency DRAM channels plus a GPU with high memory bandwidth ensure that frame data can move fast enough to sustain real-time playback of multiple 4K/8K streams.
- An NVMe-based fast tier (ideally RAID-striped) keeps storage read speed above the sustained bitrate of the footage, preventing playback stalls.
- Adequate DRAM capacity allows the editor to cache more of the timeline in memory, minimizing re-reads from storage during scrubbing and effect previews.
- Proxy/preview media workflows can be used as a software-level complement to the hardware hierarchy, trading some resolution for smoother real-time editing on lower-tier hardware.

Question 4

1. Influence of memory factors on numerical performance

- **Cache hierarchy (L1/L2/L3):** Weather models repeatedly apply the same numerical stencils (e.g., finite-difference operations) across grid points. A well-utilized cache hierarchy keeps nearby grid cells and intermediate results close to the CPU, exploiting spatial and temporal locality and avoiding costly main-memory round-trips for every arithmetic operation.
- **DRAM bandwidth:** Simulations sweep across huge 3D/4D atmospheric grids, streaming large volumes of data between DRAM and the CPU/GPU each iteration. If bandwidth is insufficient, the processor stalls waiting for data (a "memory-bound" workload), even if its raw compute (FLOPs) capacity is high.
- **Large-capacity memory:** High-resolution global or regional forecasting grids, with multiple atmospheric variables at many altitude levels and time steps, require very large datasets to remain resident in memory. Insufficient capacity forces disk paging, which is disastrous for iterative numerical solvers that revisit the same data thousands of times.

2. Why this workload is bottlenecked by memory, not just compute

Numerical weather prediction is a classic example of a memory-bandwidth-bound workload: the ratio of arithmetic operations to bytes moved (arithmetic intensity) is relatively low for many stencil computations, meaning performance is capped by how fast data can be delivered to the processor rather than how many operations it can execute.

3. Proposed enhancements

Enhancement	Benefit
Blocking/tiling algorithms matched to cache size	Reorganizes computation so each cache line is reused many times before eviction, maximizing data locality
Multi-level cache with large L3 / shared cache	Keeps larger working sets on-chip, reducing DRAM traffic for repeated stencil passes
High-bandwidth memory (HBM) or multi-channel DDR	Increases sustained GB/s delivered to compute cores, directly addressing the bandwidth bottleneck
NUMA-aware memory allocation	Ensures each processor core accesses memory local to its socket, reducing cross-node latency in large multi-socket systems
Large-capacity DRAM (hundreds of GB to TB) with fast interconnect to storage	Keeps full simulation grids resident in memory across time steps, avoiding disk-paging penalties
Prefetching and streaming I/O for dataset ingestion	Overlaps loading of new sensor/observation data with ongoing computation, hiding storage latency

4. Expected outcome

Together, these enhancements increase the fraction of time the processor spends computing rather than waiting for data. Higher cache-hit rates reduce redundant DRAM traffic, higher bandwidth removes data-delivery bottlenecks, and larger in-memory capacity avoids costly storage access mid-simulation — collectively boosting throughput for repeated large-scale numerical computations.

Question 5

1. Comparison of memory technologies

Technology	Latency	Capacity	Persistence	Power consumption
SRAM	Very low (ns)	Small (KB–MB)	Volatile	Low static power when idle, but costly per bit — used sparingly
DRAM	Low (tens of ns)	Medium–large (MB–GB)	Volatile	Higher power due to constant refresh cycles — significant drain in an always-on device
NAND Flash	Higher (μs, especially writes)	Large (GB+)	Non-volatile	Low idle power, but write operations are energy-intensive and wear out cells over time
Low-power NVM (FRAM/MRAM/ReRAM)	Low–moderate	Small–medium	Non-volatile	Very low power, no refresh needed, fast writes with minimal wear — ideal for frequent small updates

2. Workload characteristics of a smart home gateway

- Continuous, low-intensity sensor polling (motion, temperature, humidity) generates small, frequent data updates.
- Camera streams generate large bursts of data that need temporary buffering before local analysis or upload.
- The device must operate on limited energy (often battery- or low-power-supply-based), so idle/standby power matters as much as active power.
- Some data (device configuration, event logs, recent sensor history) must survive power loss or reboot.

3. Proposed memory hierarchy

- **Tier 1 — Small on-chip SRAM:** Used for the microcontroller's active registers and the immediate processing buffer for whichever sensor/appliance data is currently being handled. Extremely fast and low-latency, but kept minimal to control cost and static power.
- **Tier 2 — Low-power NVM (FRAM/MRAM) for frequently updated persistent data:** Ideal for storing frequently changing but important data — e.g., current sensor readings, device state, and short event logs — because it combines non-volatility with low write energy and near-DRAM speed, avoiding both the refresh overhead of DRAM and the wear/latency penalty of Flash.
- **Tier 3 — Small DRAM buffer (used selectively, power-gated when idle):** Provides temporary high-speed buffering for camera image/video frames during active capture or streaming. Powered down or placed in self-refresh/low-power mode when the camera is inactive, to conserve energy.

- **Tier 4 — NAND Flash for bulk, long-term storage:** Stores recorded video clips, historical sensor logs, and firmware images that must persist reliably over the long term. Writes are batched (e.g., buffering in DRAM/NVM first, then flushing to Flash) to reduce the number of energy-costly Flash write cycles and extend device lifespan.

4. How this design balances speed, persistence, and energy

- Frequently accessed, small, and persistence-sensitive data (device state, recent readings) lives in low-power NVM rather than DRAM, cutting refresh-related energy waste.
- DRAM is used only for short bursts of camera-buffering activity and power-gated the rest of the time, since it is the most power-hungry volatile tier.
- Flash is reserved for bulk, infrequent writes, extending both battery life and Flash endurance.
- This tiered approach ensures rapid access to hot, frequently used data while guaranteeing that critical information survives power interruptions, all within a tight energy budget suitable for a continuously operating smart home gateway.